



PS C:\Users\v-mar> docker run postgres -e POSTGRES_PASSWORD=123

Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value for the superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".

You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all connections without a password. This is *not* recommended.

See PostgreSQL documentation about "trust":

[<https://www.postgresql.org/docs/current/auth-trust.html>] (<https://www.postgresql.org/c>

The error message indicates that the PostgreSQL Docker container requires you to specify the superuser (postgres) password with the environment variable POSTGRES_PASSWORD when initializing the database for the first time.

Your Docker command has a small syntax issue: there is a missing `-e` flag before POSTGRES_PASSWORD=123. The correct command should explicitly use `-e` to set the environment variable.

Use this corrected command to run the PostgreSQL container with the password:

```
docker run -e POSTGRES_PASSWORD=123 postgres
```

This tells Docker to run a postgres container and set the POSTGRES_PASSWORD environment variable with the password 123 for the superuser.

Alternatively, you can also specify other useful environment variables or port mappings if needed. The basic requirement is to include `-e POSTGRES_PASSWORD=<your_password>` for the first initialization.

This will fix the error and allow PostgreSQL to start properly in the container. For more details, you can review the official PostgreSQL Docker image documentation.

ОТВЕТЬ НА РУССКОМ

```
{"queries":["docker run postgres POSTGRES_PASSWORD environment variable usage"]}
```

PS C:\Users\v-mar> docker run postgres -e POSTGRES_PASSWORD=123

Error: Database is uninitialized and superuser password is not specified.
You must specify POSTGRES_PASSWORD to a non-empty value for the superuser. For example, "-e POSTGRES_PASSWORD=password" on "docker run".

You may also use "POSTGRES_HOST_AUTH_METHOD=trust" to allow all connections without a password. This is *not* recommended.

See PostgreSQL documentation about "trust":
<https://www.postgresql.org/docs/current/auth-trust.html>

ответ на русском

Ошибка возникает, потому что при первом запуске контейнера PostgreSQL необходимо задать пароль суперпользователя через переменную окружения POSTGRES_PASSWORD. В вашем случае команда должна выглядеть так:

```
docker run -e POSTGRES_PASSWORD=123 postgres
```

Очень важно использовать ключ `-e` перед `POSTGRES_PASSWORD=123`, чтобы правильно передать переменную окружения в контейнер. Без этого Docker обрабатывает аргументы неправильно, и база не инициализируется.

Возможна также альтернатива — использовать переменную окружения `POSTGRES_HOST_AUTH_METHOD=trust`, которая позволяет запускать контейнер без пароля, но это небезопасно и не рекомендуется, особенно в продакшн-средах.

Итог: исправьте команду, добавив `-e`, пример:

```
docker run -e POSTGRES_PASSWORD=123 postgres
```

Это позволит инициализировать базу и задать пароль суперпользователя, устранив ошибку. Более подробные рекомендации есть в официальной документации PostgreSQL для Docker и в обсуждениях пользователей по данной проблеме. ^[1] ^[2] ^[3]

✱✱

я хочу запустить образ через графический интерфейс докера, где мне указать данный параметр запуска?

Для указания параметра `POSTGRES_PASSWORD` при запуске образа PostgreSQL через графический интерфейс Docker Desktop выполните следующие шаги:

1. Откройте Docker Desktop и перейдите в раздел "Containers / Apps".
2. Нажмите кнопку для создания нового контейнера ("Create" или "Run a new container").
3. Выберите образ `postgres` из списка или введите его в поле поиска.
4. В настройках контейнера найдите раздел, связанный с переменными окружения (Environment Variables).
5. Добавьте новую переменную окружения:
 - В поле "Name" укажите `POSTGRES_PASSWORD`.
 - В поле "Value" укажите желаемый пароль, например `123`.
6. Проверьте остальные настройки (порты, тома и т.д.) и запустите контейнер через интерфейс.

Таким образом, параметр запуска с паролем задается через переменную окружения в разделе Environment Variables GUI Docker Desktop. Это эквивалентно указанию `-e POSTGRES_PASSWORD=123` в командной строке.

Если нужна помощь с конкретной версией интерфейса, могу помочь с более точной инструкцией.

не могу найти кнопку создания или запуска нового контейнера

Если вы не можете найти кнопку для создания или запуска нового контейнера в графическом интерфейсе Docker Desktop, скорее всего, вы должны использовать следующий порядок действий:

1. Откройте Docker Desktop.
2. На главной странице или в меню слева выберите раздел **"Containers / Apps"**.
3. В этом разделе ищите кнопку или иконку с надписью вроде **"Run new container"**, **"Create container"** или **"+"** для создания нового контейнера. В некоторых версиях Docker Desktop она может быть расположена в правом верхнем углу.
4. Если такой кнопки нет, попробуйте в верхнем меню нажать на кнопку **"Images"**, затем найти образ `postgres` в списке загруженных образов.
5. Рядом с образом должна быть кнопка **"Run"** или похожая, которая запускает контейнер с этим образом и предлагает настроить параметры, включая переменные окружения.

6. В открывшемся окне запуска контейнера найдите раздел переменных окружения (Environment Variables) и добавьте туда POSTGRES_PASSWORD с нужным значением.

Если в вашем интерфейсе действительно нет явной кнопки создания или запуска контейнера, это может означать, что версия Docker Desktop или настройки интерфейса отличаются. В таком случае классическим и надежным вариантом будет запуск контейнера через командную строку с правильными параметрами.

Таким образом, чтобы через GUI указать параметр POSTGRES_PASSWORD, нужно сначала найти и нажать кнопку запуска контейнера рядом с нужным образом в разделе "Images" или "Containers / Apps", а затем в настройках контейнера добавить переменную окружения с паролем.

Если что-то не получается, рекомендую использовать терминал для команды:

```
docker run -e POSTGRES_PASSWORD=123 postgres
```

Это проще и гарантированно сработает во всех версиях Docker. [\[11\]](#) [\[12\]](#)



я смог запустить его, могу ли я сделать из этого контейнера свой dockerfile?

Да, из уже запущенного контейнера вы можете сделать свой Dockerfile, но обычно это делают так: в Dockerfile указывают базовый образ postgres и прописывают необходимые параметры окружения и конфигурацию. Например:

```
FROM postgres
ENV POSTGRES_USER=my_user
ENV POSTGRES_PASSWORD=my_password
ENV POSTGRES_DB=my_database
EXPOSE 5432
```

После этого вы создаёте новый образ с помощью команды `docker build -t my_postgres .` из папки с этим Dockerfile.

Если у вас есть изменения внутри запущенного контейнера, которые вы хотите сохранить в образе, можно использовать команду `docker commit tainer_id> my_postgres` для создания нового образа из текущего состояния контейнера. Однако такой подход менее контролируем и менее воспроизводим, чем написание Dockerfile.

Рекомендуется использовать Dockerfile для гибкости, удобства настройки и версионности. При этом Dockerfile можно дополнить файлами и скриптами для инициализации базы и нужной настройки.

Итого:

- Можно создать Dockerfile на базе официального postgres с нужными параметрами.
- Можно сделать образ из текущего запущенного контейнера через `docker commit`, но Dockerfile — лучший способ.
- Для создания Dockerfile достаточно базового файла с `FROM postgres` и нужными переменными окружения.

Это позволит вам создать собственный настраиваемый образ PostgreSQL на основе официального и использовать его в дальнейшем. [\[21\]](#) [\[22\]](#)

✱

я создал свой контейнер с параметрами и запустил, как мне преобразовать его в dockerfile

Если вы уже создали и запустили контейнер с нужными параметрами, то прямого способа автоматически преобразовать состояние контейнера в Dockerfile не существует, так как Dockerfile — это текстовый сценарий, описывающий инструкции сборки образа, а контейнер — результат запуска образа с изменениями в файловой системе и параметрах.

Однако есть несколько способов получить стартовую точку для создания Dockerfile:

1. Используйте команду `docker commit tainer_id <new_image_name>` для создания нового образа из текущего контейнера. Это сохранит все изменения, но в виде образа, а не Dockerfile.
2. Чтобы создать Dockerfile, необходимо вручную описать шаги, которые вы делали в контейнере, включая базовый образ (обычно `FROM postgres`), установку переменных окружения (`ENV POSTGRES_PASSWORD=...` или использование `ARG`), копирование файлов и запуск команд настройки (`RUN, CMD`).
3. Можно исследовать текущий образ с помощью `docker history <image>` — это покажет слои и команды, которые использовались для его создания. Это поможет воспроизвести аналогичный Dockerfile.
4. Создайте Dockerfile примерно такого вида:

```
FROM postgres
ENV POSTGRES_PASSWORD=your_password
# Добавьте сюда другие нужные инструкции (копирование файлов, запуск скриптов и т.д.)
```

5. Для конкретных настроек базы данных и параметров нужно добавить соответствующие команды (`RUN, COPY` и т.п.) в Dockerfile.

В итоге: полностью автоматического преобразования контейнера в Dockerfile нет, придется вручную воспроизвести логику создания образа, используя информацию о запущенном контейнере и сделанных изменениях. Команда `docker commit` поможет получить рабочий образ, но не Dockerfile. С Dockerfile же проще управлять версиями и изменениями образа. [\[31\]](#) [\[32\]](#) [\[33\]](#)



хорошо, как мне создать dockerfile с нуля

Для создания Dockerfile с нуля для PostgreSQL достаточно создать текстовый файл Dockerfile с минимальным набором инструкций. Вот пример базового Dockerfile для PostgreSQL:

```
# Используем официальный образ PostgreSQL как базовый
FROM postgres:latest

# Указываем переменные окружения для базы данных
ENV POSTGRES_USER=myuser
ENV POSTGRES_PASSWORD=mypassword
ENV POSTGRES_DB=mydatabase

# Копируем SQL скрипты инициализации (если есть) в специальную папку
# COPY ./init.sql /docker-entrypoint-initdb.d/

# Открываем порт PostgreSQL
EXPOSE 5432
```

Пояснения:

- `FROM postgres:latest` — берет официальный образ PostgreSQL последней версии.
- `ENV` задает необходимые переменные окружения для создания пользователя, пароля и базы при запуске.
- Если у вас есть SQL-скрипты для предзаполнения базы, их можно положить в папку рядом с Dockerfile и скопировать командой `COPY` в контейнер в каталог `/docker-entrypoint-initdb.d/`. Они автоматически выполнятся при инициализации.
- `EXPOSE 5432` — открывает стандартный порт для доступа к PostgreSQL.

Чтобы собрать образ из Dockerfile, выполните в терминале команду:

```
docker build -t my_postgres_image .
```

Где `.` — текущая директория с Dockerfile.

Далее можно запускать контейнер из своего образа:

```
docker run -d -p 5432:5432 --name my_postgres_container my_postgres_image
```

Так вы создадите полностью управляемый образ PostgreSQL с нужными настройками, который можно версионировать и распространять.

Если понадобится, можно добавить больше инструкций в Dockerfile для вашей специфической конфигурации.

Это базовый рабочий вариант Dockerfile для PostgreSQL с самого начала. ^[41]



PS C:\Users\v-mar> docker build -t my_postgres_image D:\облако\университет\магистратура\валидация и тестирован

ие систем

ERROR: docker: 'docker buildx build' requires 1 argument

Usage: docker buildx build [OPTIONS] PATH | URL | -

Run 'docker buildx build --help' for more information

Ошибка возникает из-за того, что путь к контексту сборки вы указали в командной строке с пробелами и кириллицей, и Docker не воспринимает его корректно. Чтобы создать Docker образ из Dockerfile с нуля, сделайте так:

1. Создайте папку, где будет ваш Dockerfile, например:

```
D:\облако\университет\магистратура\валидация_и_тестирование_систем
```

(лучше заменить пробелы в имени папки на символы подчеркивания или использовать кавычки)

2. Внутри этой папки создайте файл с именем Dockerfile и поместите туда инструкции, например:

```
FROM postgres:latest
ENV POSTGRES_USER=myuser
ENV POSTGRES_PASSWORD=mypassword
ENV POSTGRES_DB=mydatabase
EXPOSE 5432
```

3. Затем выполните команду из PowerShell, учитывая кавычки вокруг пути:

```
docker build -t my_postgres_image "D:\облако\университет\магистратура\валидация и тестирование систем"
```

Или, если папка содержит пробелы, можно перейти в неё и собрать образ из текущей директории:

```
cd "D:\облако\университет\магистратура\валидация и тестирование систем"
docker build -t my_postgres_image .
```

Точка . означает текущую директорию с Dockerfile.

Так вы избежите ошибки и успешно создадите образ из вашего Dockerfile.

Основные моменты:

- Путь до Dockerfile нужно указывать с кавычками, если в нём есть пробелы.
- Альтернативно запускайте build из директории с Dockerfile.
- В Dockerfile укажите базовый образ и переменные окружения, минимум для PostgreSQL.

Это стандартный рабочий способ для создания Docker образа с нуля.^[51]



команда сработала, что я могу с этим делать дальше?

После успешного создания Docker образа PostgreSQL у вас есть несколько вариантов дальнейших действий:

1. Запуск контейнера из вашего образа для использования базы данных:

```
docker run -d -p 5432:5432 --name my_postgres_container my_postgres_image
```

Это запустит контейнер в фоне и пробросит порт 5432 на хост, чтобы подключаться к базе данных.

2. Подключение к базе данных через любой PostgreSQL клиент или psql для создания таблиц, добавления, изменения и извлечения данных.
3. Организация постоянного хранения данных через Docker volumes (монтирование папки с данными на хосте):

```
docker run -d -p 5432:5432 -v /path/on/host:/var/lib/postgresql/data my_postgres_image
```

Это обеспечит сохранность данных при перезапуске или удалении контейнера.

4. Автоматизация с помощью docker-compose — можно создать файл docker-compose.yml для удобного управления контейнером и его настройками.
5. Создание резервных копий и восстановление данных с помощью команд pg_dump и psql прямо внутри контейнера через docker exec.
6. Дальнейшее расширение Dockerfile, добавление дополнительных скриптов и параметров для кастомизации вашей базы.

Таким образом, теперь у вас есть свой образ PostgreSQL, который можно использовать для:

- Быстрого развертывания среды разработки или тестирования.
- Автоматизированного развертывания в разных средах.

- Настройки и масштабирования через Docker.

Это стандартный рабочий цикл с Docker-образами PostgreSQL, который поможет организовать удобное и надежное использование СУБД. [\[61\]](#) [\[62\]](#) [\[63\]](#)



как я могу Создать docker-compose.yml файл для запуска нескольких контейнеров в связке (например, веб-сервер и базу данных).

Чтобы создать docker-compose.yml для запуска нескольких связанных контейнеров (например, веб-сервер и база данных PostgreSQL), можно использовать такой пример:

```
version: '3.8'

services:
  db:
    image: postgres:latest
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydatabase
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  web:
    image: your_web_server_image
    depends_on:
      - db
    ports:
      - "80:80"
    environment:
      DATABASE_URL: postgres://myuser:mypassword@db:5432/mydatabase

volumes:
  db_data:
```

Объяснение:

- `services` — здесь описываются ваши контейнеры.
- Контейнер `db` использует образ PostgreSQL, задает пользователя, пароль, базу, подключает volume для постоянного хранения данных, и пробрасывает порт.
- Контейнер `web` зависит от `db` (т.е. запускается после него), пробрасывает порт 80 и получает переменную с URL подключения к базе.
- `volumes` указан общий volume для данных базы, чтобы данные сохранялись между перезапусками.

Далее:

1. Сохраните этот YAML в файл `docker-compose.yml`.
2. Запустите команду из директории с файлом:

```
docker-compose up -d
```

3. Docker Compose автоматически создаст и запустит оба контейнера, свяжет их по имени `db`, и вы получите работающую связку веб-сервера с базой.

Этот шаблон можно настраивать, добавлять другие сервисы, сети и переменные по необходимости.

Это стандартный способ организовать многоконтейнерные приложения с помощью Docker Compose. [\[71\]](#) [\[72\]](#) [\[73\]](#)

✱

сделай рабочий пример с каким нибудь веб сервером

Вот рабочий пример `docker-compose.yml`, где используется PostgreSQL и простой веб-сервер Nginx, который отдаёт статические файлы:

```
version: '3.8'

services:
  db:
    image: postgres:latest
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydatabase
    volumes:
      - db_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  web:
    image: nginx:latest
    volumes:
      - ./html:/usr/share/nginx/html:ro
    ports:
      - "8080:80"
    depends_on:
      - db

volumes:
  db_data:
```

Инструкция для запуска:

1. Создайте папку рядом с этим файлом `docker-compose.yml`.
2. Внутри создайте подкаталог `html` и положите туда простой файл `index.html`, например:

```
<html>
  <body>
    <h1>Привет, Nginx с PostgreSQL!</h1>
  </body>
</html>
```

3. Запустите команду:

```
docker-compose up -d
```

4. Откройте в браузере <http://localhost:8080> — вы увидите страницу из `index.html`.
5. PostgreSQL будет доступен на стандартном порту 5432 и готов принимать подключения.

Таким образом вы организуете связку веб-сервер + база данных с помощью одного файла Docker Compose. Можно легко расширять, добавлять свои приложения вместо Nginx, менять настройки и управлять контейнерами централизованно.

Это простой рабочий старт для многоконтейнерного приложения с веб-сервером и базой на Docker. [\[81\]](#) [\[82\]](#)

✱

project name must not be empty

Ошибка "project name must not be empty" возникает в Docker Compose, когда имя проекта не задано явно, а Docker пытается вывести его из имени текущей директории, которая может содержать пробелы или недопустимые символы.

Для исправления:

1. Укажите имя проекта явно командой при запуске:

```
docker-compose -p myproject up -d
```

где `myproject` — валидное имя проекта (маленькие буквы, цифры, дефисы или подчеркивания).

2. Также можно задать имя проекта в файле `docker-compose.yml`, добавив сверху так:

```
name: myproject
version: '3.8'

services:
```

Однако в новых версиях Compose поддержка этого поля не всегда работает, поэтому лучше использовать ключ `-p`.

3. Более безопасный способ — переименовать папку с вашим проектом, чтобы в имени не было пробелов и спецсимволов. Например, вместо

```
D:\облако\университет\магистратура\валидация и тестирование систем
```

сделать

```
D:\oblakouniversitet\magistratura\validacia_i_testirovanie_sistem
```

Пример запуска с указанием имени проекта через CLI решит проблему с пустым именем:

```
docker-compose -p myproject up -d
```

Это гарантирует, что проект получит правильное имя и не вызовет ошибки. [\[93\]](#) [\[94\]](#)

✱

Error response from daemon: failed to set up container networking: driver failed programming external connecti

vity on endpoint test_project-db-1

(a380ff6d80b4bfe77603b911781e7aac75c9419994c1a731b2397cb1f5cc74cf): Bind for 0.0.0.0:5432 failed: port is already allocated

Ошибка "Bind for 0.0.0.0:5432 failed: port is already allocated" означает, что порт 5432 на вашем хосте уже занят другим процессом или контейнером. Вот что можно сделать:

1. Проверьте, что занимает порт 5432:

- На Windows в PowerShell выполните:

```
netstat -aon | findstr :5432
```

Затем найдите PID процесса и закончите его в диспетчере задач, если это необходимо.

- Или используйте команду Docker, чтобы увидеть запущенные контейнеры:

```
docker ps
```

и остановите контейнер, который уже использует этот порт:

```
docker stop tainer_id>
```

2. Если удалить текущий процесс, занимающий порт, нельзя, измените проброс порта в вашем `docker-compose.yml`. Например, замените:

```
ports:
  - "5432:5432"
```

на

```
ports:
  - "5433:5432"
```

Таким образом, внутренний порт контейнера останется 5432, а снаружи вы будете использовать 5433.

3. Перезапустите контейнеры с обновленным `docker-compose.yml`.

Это самый распространенный и надежный способ решить проблему с портом, занятым другим процессом или контейнером. [\[103\]](#) [\[104\]](#) [\[105\]](#)

✱

всё заработало, но на локалхосте я не вижу страницы

Если страница не отображается при обращении к локальному `localhost:8080` (или другому порту), проверьте следующие моменты:

1. Проверьте, что контейнер с `nginx` действительно запущен командой:

```
docker ps
```

Убедитесь, что в колонке Ports указано "0.0.0.0:8080→80/tcp" (если у вас порт 8080 проброшен).

2. Проверьте, что в вашем локальном каталоге `html` есть файл `index.html` и он доступен для чтения. Например, путь должен совпадать с монтируемым в контейнер:

```
volumes:
  - ./html:/usr/share/nginx/html:ro
```

3. Проверьте права на файлы в каталоге `html`; `nginx` должен иметь к ним доступ.
4. Проверьте логи контейнера командой:

```
docker logs tainer_id или имя контейнера nginx>
```

Если есть ошибки при загрузке страницы или в конфигурации `nginx` — они будут в логах.

5. Убедитесь, что браузер не блокирует подключение, попробуйте открыть страницу другим браузером или в режиме инкогнито.

6. Если вы используете Windows, проверьте настройки фаервола — возможно он блокирует порт 8080.

7. Можно подключиться внутрь контейнера и проверить содержимое директории:

```
docker exec -it <nginx_container_name> sh  
ls /usr/share/nginx/html
```

8. Если нужны дополнительные настройки nginx, можно создать свой конфигурационный файл и смонтировать его в контейнер.

Эти проверки обычно помогают выявить причину, почему не отображается страница с nginx внутри Docker. Скорее всего, проблема в путях монтирования или запущен не тот контейнер с нужным контентом. [\[113\]](#) [\[114\]](#) [\[115\]](#)

✱

Error: in 18+, these Docker images are configured to store database data in a

```
format which is compatible with "pg_ctlcluster" (specifically, using  
major-version-specific directory names). This better reflects how  
PostgreSQL itself works, and how upgrades are to be performed
```

Начиная с версии 18+ официальные Docker образы PostgreSQL изменили схему хранения данных, чтобы поддерживать совместимость с утилитой `pg_ctlcluster`. Это отражает внутреннюю логику работы PostgreSQL и облегчает выполнение обновлений кластера.

Что важно понимать:

- В новых образах данные базы хранятся в каталогах, специфичных для мажорной версии PostgreSQL (например, `main`, `12`, `13` и т.д.), что связано с управлением кластерами через `pg_ctlcluster`.
- Это изменение улучшает процесс управления версиями и обновлениями PostgreSQL внутри контейнеров.

Если вы видите сообщение об этом, то, скорее всего, ваш контейнер использует новую структуру данных. Чтобы корректно работать с такими образами и данными, рекомендуется:

- Использовать официальные инструменты и команды PostgreSQL для управления кластерами данных внутри контейнера.
- При необходимости обновления базы следовать официальной документации по работе с `pg_ctlcluster`.
- Убедиться, что `volume` или папка с данными контейнера корректно смонтирована, и что кластеры данных не повреждены.

Таким образом, это не ошибка, а информационное сообщение о смене способа хранения данных в новых образах PostgreSQL Docker, что помогает администрированию и обновлению базы данных. [\[123\]](#) [\[124\]](#)

✱

да, но это не позволяет запустить базу данных в составе контейнера

Проблема запуска базы данных PostgreSQL 18+ в контейнере чаще всего связана с изменившимся путём хранения данных и необходимостью корректно указать переменную окружения PGDATA для правильной работы кластера.

Рекомендации для успешного запуска:

1. В docker-compose.yml или при запуске контейнера укажите переменную окружения, указывающую на актуальный каталог данных в контейнере, например:

```
environment:
  - POSTGRES_USER=your_user
  - POSTGRES_PASSWORD=your_password
  - POSTGRES_DB=your_db
  - PGDATA=/var/lib/postgresql/data/pgdata
```

2. Обеспечьте корректное монтирование volume для каталога /var/lib/postgresql/data на хосте, где хранятся данные.
3. Если используется старый volume с данными, несовместимый с новой версией PostgreSQL, лучше создать новый volume или очистить старый, чтобы база инициализировалась заново.
4. Проверьте логи контейнера `docker logs tainer_id` — там будут сообщения, связанные с ошибками запуска.
5. Убедитесь, что в томе нет повреждённых или устаревших данных от предыдущих версий.

Пример службы PostgreSQL с указанием PGDATA:

```
services:
  db:
    image: postgres:18
    environment:
      POSTGRES_USER: myuser
      POSTGRES_PASSWORD: mypassword
      POSTGRES_DB: mydatabase
      PGDATA: /var/lib/postgresql/data/pgdata
    volumes:
      - pgdata:/var/lib/postgresql/data
    ports:
      - "5432:5432"
```

```
volumes:
  pgdata:
```

Если проблема сохраняется, попробуйте удалить volume с прежними данными или используйте новый volume.

Таким образом, для запуска PostgreSQL 18+ важно учитывать новую структуру хранения данных с каталогами, совместимыми с `pg_ctlcluster`, и правильно указывать переменную `PGDATA` в настройках контейнера. [\[133\]](#) [\[134\]](#) [\[135\]](#)

✱✱

я увидел в каталоге `/usr/share/nginx/html` свой файл но на локалхосте он не открывается

Если файл `index.html` действительно присутствует в контейнере `nginx` по пути `/usr/share/nginx/html`, но страница не открывается на `localhost`, вероятные причины и решения следующие:

1. Проверьте, что порт `nginx` проброшен правильно в `docker-compose.yml` или команде запуска:

```
ports:
  - "8080:80"
```

Тогда в браузере нужно открывать `http://localhost:8080`.

2. Убедитесь, что контейнер `nginx` запущен и не падает:

```
docker ps
docker logs <nginx_container_id>
```

3. Проверьте настройки `firewall` и антивируса, чтобы они не блокировали порт `8080`.
4. Убедитесь, что в вашей системе нет другого процесса, который уже занимает порт `8080`:

```
netstat -aon | findstr 8080
```

5. Проверьте содержимое каталога `html` на хосте, убедитесь, что там действительно есть `index.html`, и этот каталог смонтирован в `nginx` как том:

```
volumes:
  - ./html:/usr/share/nginx/html:ro
```

Также права доступа к файлам должны позволять `nginx` читать эти файлы.

6. Можно попробовать подключиться внутрь контейнера и запустить `curl`:

```
docker exec -it <nginx_container_id> sh
```



```
curl http://localhost
```

Если ответ есть — nginx работает, проблема возможно на стороне сети хоста.

7. Проверьте nginx конфигурацию, возможно, она перезаписана и не обслуживает нужный каталог.

Если все вышеперечисленное проверено и исправлено, страница должна открываться по указанному порту.

Это стандартные шаги по диагностике ситуаций, когда nginx в docker видит файлы, но браузер не показывает страницу. [\[143\]](#) [\[144\]](#) [\[145\]](#)

**

там на странице проблема с кодировкой

Проблемы с кодировкой на странице, отдаваемой nginx в Docker, часто возникают из-за отсутствия или неправильной настройки заголовка Content-Type с указанием кодировки в конфигурации сервера.

Для исправления в конфигурации nginx (обычно `/etc/nginx/nginx.conf` или в файлах сайта), нужно добавить или изменить директиву charset и удостовериться, что заголовок Content-Type включает UTF-8. Например:

```
http {
    ...
    charset utf-8;
    ...
    server {
        ...
        location / {
            root /usr/share/nginx/html;
            index index.html;
            default_type text/html;
            add_header Content-Type "text/html; charset=utf-8";
        }
    }
}
```

Если вы используете стандартный образ nginx из Docker Hub и не меняли конфигурацию, можно создать свой файл `nginx.conf` с нужными параметрами и примонтировать его в контейнер:

```
services:
  web:
    image: nginx:latest
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
```

```
ports:
  - "8080:80"
```

Также проверьте, что ваш HTML-файл содержит `<meta charset="UTF-8">` в секции `<head>`, это

```
<head>
  <meta charset="UTF-8">
  <title>Пример страницы</title>
</head>
```

Таким образом, настройка заголовков Content-Type с кодировкой UTF-8 в конфиге nginx и корректная декларация в HTML решит проблему с неправильной кодировкой на странице. [\[153\]](#) [\[154\]](#) [\[155\]](#)

✱✱

как создать пользовательскую сеть в docker

Чтобы создать пользовательскую сеть в Docker, используйте команду:

```
docker network create --driver bridge my_custom_network
```

Где:

- `--driver bridge` — создаёт пользовательскую мостовую сеть (обычно используется для изоляции контейнеров внутри одного хоста),
- `my_custom_network` — имя вашей новой сети.

После создания пользовательской сети вы можете запускать контейнеры с подключением к ней:

```
docker run --network my_custom_network my_image
```

Или в `docker-compose.yml` укажите сеть:

```
networks:
  my_custom_network:
    driver: bridge

services:
  web:
    image: nginx
    networks:
      - my_custom_network

  db:
    image: postgres
```

```
networks:
  - my_custom_network
```

Запускаете через:

```
docker-compose up -d
```

Пользовательские сети позволяют лучше управлять связью контейнеров и повышают изоляцию по сравнению с сетью по умолчанию.

Это стандартный способ организации сетевого взаимодействия между контейнерами в Docker. [\[163\]](#) [\[164\]](#) [\[165\]](#)

✱

я выполнил команду `docker run --network my_custom_network my_image`

как протестировать связь между контейнерами в сети.

Чтобы протестировать связь между контейнерами в одной пользовательской сети Docker, выполните следующие шаги:

1. Запустите два контейнера, подключенных к одной сети, например `my_custom_network`:

```
docker run -dit --name container1 --network my_custom_network alpine sh
docker run -dit --name container2 --network my_custom_network alpine sh
```

2. Используйте команду `ping` из одного контейнера к другому для проверки сети:

```
docker exec container1 ping -c 4 container2
```

Если связь есть, вы увидите ответы типа:

```
64 bytes from container2: icmp_seq=1 ttl=64 time=0.1 ms
```

3. Можно также проверить с помощью `docker exec` подключение по TCP к нужному порту (например, к базе данных) с помощью `telnet` или установленных инструментов в контейнере.
4. Проверить IP адреса контейнеров можно командой:

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' container_name
```

Таким образом, проверка с помощью `ping` — самый простой способ убедиться, что два контейнера успешно связаны и могут общаться в одной сети Docker.

Если `ping` недоступен в образе (например, в `alpine` может не быть установлен), установите его:

```
docker exec container1 apk add iputils
```

Это стандартный практический подход для тестирования связи между Docker контейнерами в одинаковой сети. [\[173\]](#) [\[174\]](#)

✱

моя сеть называется `test_network` сформируй команду на запуск контейнеров

Если ваша пользовательская сеть называется `test_network`, то команда запуска контейнеров с подключением к этой сети будет такой:

Для первого контейнера:

```
docker run -dit --name container1 --network test_network alpine sh
```

Для второго контейнера:

```
docker run -dit --name container2 --network test_network alpine sh
```

Где:

- `-d` — запускает контейнер в фоне,
- `-i` — позволяет работать с интерактивным вводом,
- `-t` — создает терминал,
- `--name` — задает имя контейнера,
- `--network test_network` — подключает контейнер к вашей пользовательской сети.

Так вы запустите контейнеры в одной сети `test_network`, и они смогут общаться друг с другом по именам контейнеров.

Для проверки сети можно внутри контейнера сделать:

```
docker exec container1 ping container2
```

Если нужно запустить конкретные образы (например, `nginx` и `postgres`), замените `alpine` на нужный образ и добавьте необходимые параметры.

Это стандартный и удобный способ запуска контейнеров в одной пользовательской сети Docker с дальнейшей проверкой связи между ними. [\[183\]](#) [\[184\]](#)



что такое alpine?

Alpine — это легковесный (около 5 МБ) дистрибутив Linux, часто используемый в Docker-контейнерах из-за своей компактности и безопасности. Он основан на musl libc и BusyBox, что делает его минималистичным и быстрым для загрузки и запуска. Alpine идеально подходит для микросервисов и приложений, где важны размеры образов и скорость развёртывания.

В Docker его используют как базовый образ для создания своих контейнеров с минимальным размером, чтобы уменьшить общий объём и ускорить работу.

Преимущества Alpine в Docker:

- Очень маленький размер образа.
- Быстрая загрузка и запуск.
- Меньше уязвимостей из-за минимального набора пакетов.
- Гибкость, позволяющая добавлять только необходимые компоненты через менеджер пакетов apk.

Пример в Dockerfile:

```
FROM alpine:latest
RUN apk add --no-cache bash
CMD ["bash"]
```

Это создаёт минималистичный контейнер с поддержкой bash на базе Alpine Linux.

Alpine — популярный выбор для оптимизации Docker-образов и быстрой работы контейнеров. [\[193\]](#) [\[194\]](#) [\[195\]](#)



почему у меня ошибка что этот контейнер уже используется

Ошибка "контейнер уже используется" возникает потому, что имя контейнера, которое вы пытаетесь назначить при запуске (с помощью параметра `--name`), уже занято существующим контейнером — работающим или остановленным.

Решения:

1. Посмотрите список всех контейнеров, включая остановленные:

```
docker ps -a
```

Найдите контейнер с именем, которое вы хотите использовать.

2. Если этот контейнер вам не нужен, удалите его:

```
docker rm <container_name_or_id>
```

Если контейнер запущен — сначала его остановите:

```
docker stop <container_name_or_id>
```

3. Если не хотите удалять, запустите новый контейнер с уникальным именем:

```
docker run --name unique_name ...
```

Таким образом, имена контейнеров в Docker должны быть уникальны. Попытка создать новый контейнер с именем, уже существующим в системе, вызовет ошибку. Либо удаляйте старый контейнер, либо задавайте другое имя для нового. [\[203\]](#) [\[204\]](#) [\[205\]](#) [\[206\]](#)

✱

• Создать том Docker.

- Подключить том к контейнеру для хранения данных.
- Протестировать сохранение данных между запусками контейнера.

Чтобы создать том Docker, подключить его к контейнеру и протестировать сохранение данных между запусками, выполните следующие шаги:

1. Создайте том:

```
docker volume create my_data_volume
```

2. Запустите контейнер с подключением тома к нужной папке, например, к каталогу для хранения данных базы:

```
docker run -d --name my_postgres_container -v my_data_volume:/var/lib/postgresql/data -e
```

3. Проверьте работу базы, добавьте в неё данные.

4. Остановите контейнер:

```
docker stop my_postgres_container
```

5. Удалите контейнер (но том останется):

```
docker rm my_postgres_container
```

6. Запустите новый контейнер с тем же томом:

```
docker run -d --name my_postgres_container_2 -v my_data_volume:/var/lib/postgresql/data
```

7. Подключитесь к новому контейнеру и проверьте, что данные сохранились.

Таким образом, том Docker обеспечивает постоянное хранение данных вне контейнера и позволяет сохранять информацию между перезапусками или удалениями контейнеров.

Это стандартный и рекомендованный способ работы с данными в Docker. [\[213\]](#) [\[214\]](#) [\[215\]](#)

✱

сделай мне запрос для проверки базы данных в этом случае, например создание таблицы и заполнение пары строк

Вот пример SQL-запроса для проверки базы данных: создание таблицы и добавление в неё пары строк.

```
CREATE TABLE IF NOT EXISTS users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(50),  
  age INT  
);  
  
INSERT INTO users (name, age) VALUES  
( 'Alice', 30),  
( 'Bob', 25);  
  
-- Проверка содержимого таблицы  
SELECT * FROM users;
```

Объяснение:

- Создаётся таблица `users` с тремя колонками: `id` (автоинкрементный ключ), `name` (строка до 50 символов), `age` (целочисленное значение).
- Вставляются две строки с именами и возрастом.
- Запрос `SELECT` выводит все записи из таблицы.

Этот запрос можно выполнить, подключившись к контейнеру PostgreSQL и вызвав `psql`:

```
docker exec -it my_postgres_container psql -U postgres -d mydatabase -c "<ваш_запрос>"
```

Или через интерактивную сессию:

```
docker exec -it my_postgres_container psql -U postgres -d mydatabase
```

затем вводить SQL-запросы напрямую.

Такой простой тест поможет проверить, что база работает и данные сохраняются.^[223]
^[224]



Error response from daemon: container 47fc6067c70da062465075424b78924bc61f8d44950f5c528442df1b8a112d4f is not running

Ошибка "container is not running" означает, что вы пытаетесь выполнить команду (например, `docker exec`) в контейнере, который в данный момент остановлен.

Что делать:

1. Проверьте статус контейнеров командой:

```
docker ps -a
```

В колонке STATUS будет указано, запущен контейнер или нет.

2. Если контейнер остановлен (exited), запустите его:

```
docker start <container_id_or_name>
```

3. После этого можно выполнять команды в контейнере, например:

```
docker exec -it <container_id_or_name> bash
```

4. Если контейнер сразу останавливается после запуска, посмотрите логи, чтобы понять причину:

```
docker logs <container_id_or_name>
```

Обычно контейнер останавливается, если основное приложение внутри него завершилось с ошибкой или отсутствует долгоживущая команда.

Это стандартный способ диагностики и решения проблемы с неработающим контейнером.
^[233] ^[234]



мне нужно запустить postgres контейнер с атрибутом PGDATA: /var/lib/postgresql/data/pgdata

Для запуска контейнера PostgreSQL с использованием переменной окружения PGDATA с значением /var/lib/postgresql/data/pgdata и монтированием тома для сохранения данных, используйте следующую команду:

```
docker run -d \
  --name my_postgres_container \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e PGDATA=/var/lib/postgresql/data/pgdata \
  -v my_postgres_data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres
```

Объяснение:

- `-e POSTGRES_PASSWORD` — задаёт пароль для пользователя postgres.
- `-e PGDATA` — указывает PostgreSQL, что данные базы хранятся в /var/lib/postgresql/data/pgdata.
- `-v my_postgres_data:/var/lib/postgresql/data` — монтирует Docker том my_postgres_data в каталог данных внутри контейнера (важно монтировать именно /var/lib/postgresql/data).
- `-p 5432:5432` — пробрасывает порт PostgreSQL наружу.

Таким образом, PostgreSQL будет использовать каталог /var/lib/postgresql/data/pgdata внутри контейнера для хранения данных, при этом данные будут сохраняться в Docker томе my_postgres_data вне контейнера на хосте. Это обеспечивает сохранность данных между запусками контейнера.

Это рекомендованный способ запуска с пользовательским каталогом данных PGDATA и Docker томом для устойчивого хранения. [\[243\]](#) [\[244\]](#) [\[245\]](#)

✱

а если я хочу отдельный том?

Если нужно использовать отдельный том для папки данных PostgreSQL в Docker с пользовательским каталогом данных PGDATA=/var/lib/postgresql/data/pgdata, то создайте отдельный том и монтируйте его именно в общий каталог данных, а внутри контейнера PostgreSQL уже будет использовать подкаталог pgdata.

Пример команды:

```
# Создать том для хранения данных
docker volume create my_pgdata_volume

# Запустить контейнер с подключением тома
docker run -d \
  --name my_postgres_container \
  -e POSTGRES_PASSWORD=mysecretpassword \
  -e PGDATA=/var/lib/postgresql/data/pgdata \
  -v my_pgdata_volume:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres
```

Объяснение:

- Том `my_pgdata_volume` примонтирован к `/var/lib/postgresql/data` (каталогу, где PostgreSQL хранит данные).
- Переменная `PGDATA` указывает структуру папок внутри контейнера, где будут располагаться основные данные — `pgdata`.
- Данными управляет PostgreSQL согласно значению `PGDATA`, но данные при этом сохраняются в том Docker на хостовой машине.
- Этот подход обеспечивает изоляцию данных от контейнера и их сохранность между запусками.

Если хотите монтировать именно подкаталог `pgdata` томом, то путь монтирования должен совпадать с этим каталогом, но такой подход менее распространён.

Рекомендуется монтировать том в `/var/lib/postgresql/data` и указать `PGDATA` для правильной работы PostgreSQL с новыми образами.

Так вы правильно организуете хранение данных в отдельном Docker томе с учётом параметра `PGDATA`. [\[253\]](#) [\[254\]](#) [\[255\]](#)



CREATE TABLE IF NOT EXISTS users (

```
id SERIAL PRIMARY KEY,
name VARCHAR(50),
age INT
);
```

```
INSERT INTO users (name, age) VALUES
('Alice', 30),
('Bob', 25);
```

```
-- Проверка содержимого таблицы
SELECT * FROM users;
```

надо ещё создать ДБ

Для создания базы данных перед выполнением запросов по созданию таблицы и вставке данных, можно выполнить SQL-команду:

```
CREATE DATABASE mydatabase;
```

Пример полного набора команд для PostgreSQL, если вы подключаетесь к серверу postgres (по умолчанию существует база postgres):

```
CREATE DATABASE mydatabase;

\c mydatabase -- переключение на новую базу

CREATE TABLE IF NOT EXISTS users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50),
    age INT
);

INSERT INTO users (name, age) VALUES
('Alice', 30),
('Bob', 25);

SELECT * FROM users;
```

Чтобы выполнить эти команды, можно запустить клиент внутри контейнера:

```
docker exec -it my_postgres_container psql -U postgres -d postgres
```

Затем в интерактивной сессии выполнить команды создания базы и таблицы.

Или все команды можно выполнить в одной строке, передав их через параметр `-c`:

```
docker exec -it my_postgres_container psql -U postgres -d postgres -c "CREATE DATABASE my
docker exec -it my_postgres_container psql -U postgres -d mydatabase -c "
CREATE TABLE IF NOT EXISTS users (id SERIAL PRIMARY KEY, name VARCHAR(50), age INT);
INSERT INTO users (name, age) VALUES ('Alice', 30), ('Bob', 25);
"
```

Это классический способ создания базы и наполнения данными в PostgreSQL, включая работу с Docker контейнером. [\[265\]](#) [\[266\]](#) [\[267\]](#)



как просмотреть данные внутри таблицы

Чтобы просмотреть данные внутри таблицы в PostgreSQL, используйте SQL-запрос:

```
SELECT * FROM users;
```

Этот запрос вернёт все строки и столбцы из таблицы `users`.

Если вы подключены к контейнеру PostgreSQL через `psql`, просто выполните этот запрос в интерактивной сессии:

```
docker exec -it my_postgres_container psql -U postgres -d mydatabase
```

В интерфейсе `psql` введите команду:

```
SELECT * FROM users;
```

Чтобы увидеть список всех таблиц в текущей базе, используйте команду `psql`:

```
\dt
```

Таким образом можно проверить содержимое таблиц и убедиться, что данные сохранены и доступны.

Если хотите выбрать конкретные столбцы или использовать условия, можно использовать:

```
SELECT name, age FROM users WHERE age > 25;
```

Это базовые способы просмотра данных в таблицах PostgreSQL. [\[275\]](#) [\[276\]](#) [\[277\]](#)



1. https://www.reddit.com/r/docker/comments/gl2f1l/database_is_uninitialized_and_superuser_password/
2. <https://ru.stackoverflow.com/questions/1204912/docker-образ-postgres-не-подключается-к-базе-и-выдает-ошибку-database-is-unini>
3. <https://stackoverflow.com/questions/60420940/how-to-fix-error-error-database-is-uninitialized-and-superuser-password-is-not>
4. <https://stackoverflow.com/questions/63262613/docker-postgres-database-is-uninitialized-and-superuser-password-is-not-specified>
5. <https://stackoverflow.com/questions/73372725/docker-runs-postgresql-in-trust-mode>
6. <https://stackoverflow.com/questions/72169310/docker-postgresql-trust-or-password>
7. https://www.reddit.com/r/docker/comments/nbi0q4/dockercompose_postgres_1_error_database_is/
8. <https://forums.docker.com/t/cant-run-postgres-container-error-database-is-uninitialized-and-superuser-password-is-not-specified/134487>

9. <https://gitlab.com/gitlab-org/gitlab/-/issues/386617>
10. <https://community.n8n.io/t/problems-with-the-postgres-image/113224>
11. <https://my-js.org/docs/guide/docker>
12. <https://learn.microsoft.com/ru-ru/virtualization/windowscontainers/quick-start/run-your-first-container>
13. <https://www.youtube.com/watch?v=lr1rYnUubpQ>
14. <https://selectel.ru/blog/tutorials/docker-desktop-windows/>
15. <https://serverspace.ru/support/help/konfiguracziya-docker-desktop-na-windows/>
16. <https://skillbox.ru/media/code/kak-rabotaet-docker-podrobnyy-gayd-ot-tekhilda/>
17. <https://stepik.org/lesson/1325756/step/1>
18. https://redos.red-soft.ru/base/redos-7_3/7_3-administation/7_3-containers/7_3-docker-install/
19. https://1cloud.ru/help/docker/docker_container
20. <https://itdoxy.com/панель-управления-docker/>
21. <https://foxminded.ua/ru/docker-postgres/>
22. <https://ru.stackoverflow.com/questions/1414297/Как-создать-docker-образ-из-существующей-postgres-бд>
23. https://www.nic.ru/help/polnoe-rukovodstvo-po-ustanovke-i-nastrojke-postgresql-v-docker_11679.html
24. <https://dockerhosting.ru/blog/postgresql-v-docker/>
25. <http://onreader.mdl.ru/DockerRailsDevelopersApplicationsEverywhere/content/Ch06.html>
26. https://help.1forma.ru/Maintenance/tech_req_1f_prepare_postgresql_docker.htm
27. <https://habr.com/ru/articles/578744/>
28. https://alley-science.ru/domains_data/files/EJanuary/METODIKA_SOZDANIYa DOCKER-OBRAZA POSTGRESQLV SREDE UBUNTULINUX.pdf
29. https://www.reddit.com/r/docker/comments/8hym1y/how_to_populate_docker_container_with_preexisting/
30. <https://ivan-shamaev.ru/docker-compose-tutorial-container-image-install/>
31. <https://cloud.ru/blog/chto-takoye-dockerfile>
32. <https://blog.trukhin.com/погружаемся-в-docker-dockerfile-и-коммуникация-между-контейнерами-e3e79fb76bb7>
33. <https://ru.stackoverflow.com/questions/1049395/Генерация-dockerfile-для-уже-существующего-контейнера>
34. https://www.youtube.com/watch?v=_pmbcKuxNkk
35. https://1cloud.ru/help/docker/dockerfile_work
36. <https://selectel.ru/blog/tutorials/how-to-create-docker-image/>
37. <https://labex.io/ru/tutorials/docker-how-to-efficiently-update-docker-images-with-dockerfile-411665>
38. <https://habr.com/ru/companies/ruvds/articles/439980/>
39. <https://learn.microsoft.com/ru-ru/virtualization/windowscontainers/manage-docker/manage-windows-dockerfile>
40. https://www.reddit.com/r/devops/comments/qa5p1b/is_there_a_way_to_modify_a_docker_container_and/
41. <https://habr.com/ru/articles/578744/>

42. <https://purpleschool.ru/knowledge-base/article/pgadmin>
43. <https://polyakovdmityy.ru/postgresql-docker-backup/>
44. <https://timeweb.cloud/tutorials/postgresql/postgresql-docker-setup>
45. <https://selectel.ru/blog/postgresql-docker-setup/>
46. https://alley-science.ru/domains_data/files/EJanuary/METODIKA_SOZDANIYa DOCKER-OBRAZA POSTGRESQLV SREDE UBUNTULINUX.pdf
47. https://www.nic.ru/help/polnoe-rukovodstvo-po-ustanovke-i-nastroyke-postgresql-v-docker_11679.html
48. <https://www.youtube.com/watch?v=dyUpyUErKfE>
49. <https://prohomelab.com/posts/postgres-p-1/>
50. <https://docs.github.com/ru/actions/tutorials/use-containerized-services/create-postgresql-service-containers>
51. <https://phoenixnap.com/kb/docker-build>
52. <https://stackoverflow.com/questions/34012094/how-to-build-dockerfile>
53. <https://www.geeksforgeeks.org/devops/docker-build/>
54. <https://docs.docker.com/get-started/docker-concepts/building-images/build-tag-and-publish-an-image/>
55. <https://docs.docker.com/reference/cli/docker/buildx/build/>
56. <https://www.cherryservers.com/blog/docker-build-command>
57. <https://stackify.com/docker-build-a-beginners-guide-to-building-docker-images/>
58. <https://stackoverflow.com/questions/45141402/build-and-run-dockerfile-with-one-command>
59. <https://www.educative.io/answers/what-is-the-docker-build-command>
60. <https://www.servermania.com/kb/articles/docker-build-command>
61. <https://habr.com/ru/companies/first/articles/683874/>
62. <https://mhost.by/knowledgebase/285/docker-ustanovka-i-nastroika-postgresql.html>
63. <https://foxminded.ua/ru/docker-postgres/>
64. <https://selectel.ru/blog/postgresql-docker-setup/>
65. https://alley-science.ru/domains_data/files/EJanuary/METODIKA_SOZDANIYa DOCKER-OBRAZA POSTGRESQLV SREDE UBUNTULINUX.pdf
66. https://help.sweb.ru/ustanovka-i-nastroyka-postgresql-v-docker_1387.html
67. https://help.1forma.ru/Maintenance/tech_req_1f_prepare_postgresql_docker.htm
68. <https://timeweb.cloud/tutorials/postgresql/postgresql-docker-setup>
69. <https://habr.com/ru/articles/578744/>
70. <https://ru.stackoverflow.com/questions/1414297/Как-создать-docker-образ-из-существующей-postgres-бд>
71. <https://www.dmosk.ru/scripts.php?object=docker-compose-webserver>
72. <https://habr.com/ru/articles/735274/>
73. <https://doka.guide/tools/docker-compose/>
74. <https://ru.stackoverflow.com/questions/1191130/docker-compose-файл-для-базы-данных>
75. <https://habr.com/ru/articles/670938/>

76. <https://learn.microsoft.com/ru-ru/dotnet/architecture/microservices/multi-container-microservice-net-applications/database-server-container>
77. <https://selectel.ru/blog/docker-compose/>
78. <https://thehost.ua/wiki/administration/docker-compose-mysql>
79. https://ru.hexlet.io/courses/docker-basics/lessons/docker-compose/theory_unit
80. https://www.reddit.com/r/docker/comments/q1zodr/how_to_connect_docker_compose_to_a_database_on/
81. <https://www.dmosk.ru/scripts.php?object=docker-compose-webserver>
82. <https://habr.com/ru/articles/735274/>
83. <https://dev.to/abraaoduarte/simple-node-application-with-docker-nginx-postgres-docker-compose-16p3>
84. <https://postgres.men/database/postgresql/4064-ru/>
85. <https://6.docs.plone.org/install/containers/examples/nginx-volto-plone-postgresql.html>
86. <https://github.com/DanielNery/odoo-docker-compose-nginx-postgresql>
87. <https://testdriven.io/blog/dockerizing-django-with-postgres-gunicorn-and-nginx/>
88. https://docker-fastapi-projects.readthedocs.io/en/latest/nginx_with_postgresql.html
89. <https://proglib.io/p/prochnyy-fundament-dlya-api-docker-node-js-nginx-postgres-2022-10-05>
90. <https://docs.docker.com/reference/samples/postgres/>
91. <https://alexaitken.nz/blog/getting-docker-react-net-core-postgres-and-nginx-playing-nice/>
92. <https://docs.docker.com/reference/samples/nginx/>
93. <https://github.com/docker/compose/issues/9519>
94. <https://docs.docker.com/compose/how-tos/project-name/>
95. <https://community.traefik.io/t/changing-project-name-in-compose-file-causes-issues-with-traefik/19399>
96. https://www.reddit.com/r/docker/comments/1mnns49/invalid_project_name_help/
97. <https://www.kubeblogs.com/how-to-avoid-issues-with-docker-compose-due-to-same-folder-names-project-isolation-best-practices/>
98. <https://github.com/earthly/earthly/issues/3164>
99. https://www.reddit.com/r/docker/comments/1dsl3q6/docker_compose_logs_missing_project_name/
100. <https://stackoverflow.com/questions/58303783/how-run-docker-compose-without-project-name>
101. <https://forums.docker.com/t/docker-compose-confilct-default-container-name-already-in-use/138556>
102. <https://www.answeroverflow.com/m/1253436370702303344>
103. <https://github.com/docker/compose/issues/4950>
104. <https://stackoverflow.com/questions/38249434/docker-postgres-failed-to-bind-tcp-0-0-0-5432-address-already-in-use>
105. <https://logfetch.com/docker-port-allocated/>
106. https://www.reddit.com/r/javahelp/comments/15cs3jb/dockercompose_error_port_5432_refused/
107. <https://github.com/prowler-cloud/prowler/issues/7684>
108. <https://forums.docker.com/t/port-is-already-allocated-when-attempting-to-run-a-container/47301>
109. <https://stackoverflow.com/questions/42635391/docker-postgres-port-is-already-allocated>

110. <https://forums.docker.com/t/postgresql-error-on-docker/68787>
111. https://www.reddit.com/r/docker/comments/18sr2bp/port_already_allocated_error/
112. <https://qna.habr.com/q/892835>
113. <https://selectel.ru/blog/install-nginx/>
114. <https://ru.stackoverflow.com/questions/903756/docker-с-контейнером-nginx-не-открывается-сайт>
115. <https://dockerhosting.ru/blog/nginx-v-docker/>
116. <https://rabota.dev/forum/threads/Не-могу-достучаться-до-nginx-в-контейнере-docker.2565/>
117. https://www.reddit.com/r/docker/comments/15q9ojh/how_do_i_display_my_index_html_file_using_nginx/
118. <https://qna.habr.com/q/722211>
119. <https://firstvds.ru/technology/ustanovka-i-nastroyka-nginx>
120. https://www.reddit.com/r/docker/comments/1f7h4kh/total_newbie_struggling_with_docker_and_nginx/
121. https://qna.habr.com/q/1383090?from=questions_similar
122. <https://ru.stackoverflow.com/questions/699720/Не-запускается-nginx-в-контейнере-docker?rq=1>
123. <https://github.com/NginxProxyManager/nginx-proxy-manager/issues/4811>
124. <https://www.postgresql.org/docs/current/app-pg-ctl.html>
125. <https://severalnines.com/blog/how-deploy-postgresql-docker-container-using-clustercontrol/>
126. <https://www.answeroverflow.com/m/1430836224620232785>
127. <https://github.com/docker-library/postgres/issues/153>
128. <https://hub.docker.com/r/paunin/postgresql-cluster-pgsq1>
129. <https://www.crunchydata.com/blog/an-easy-recipe-for-creating-a-postgresql-cluster-with-docker-swarm>
130. <https://stackoverflow.com/questions/31645550/postgresql-why-psql-cant-connect-to-server>
131. <https://hub.docker.com/r/bitnami/postgresql>
132. https://documentation.commvault.com/v11/software/getting_started_with_postgresql_in_docker_cluster.html
133. <https://habr.com/ru/articles/578744/>
134. https://www.reddit.com/r/docker/comments/1o5gdmy/postgres_18_how_do_volumes_work/
135. <https://www.cyberforum.ru/emulators/thread2871529.html>
136. <https://www.youtube.com/watch?v=UQKteGzIKG4>
137. https://help.sweb.ru/ustanovka-i-nastrojka-postgresql-v-docker_1387.html
138. <https://ru.stackoverflow.com/questions/1570401/Не-получается-запустить-docker-с-базой-данных-postgresql-16>
139. https://www.reddit.com/r/PostgreSQL/comments/1nr7ir2/postgres_18_docker_error/
140. https://www.reddit.com/r/unRAID/comments/1nrxe3f/postgres_container_wont_run/
141. <https://www.linux.org.ru/forum/admin/17732391>
142. <https://habr.com/ru/sandbox/229446/>
143. <https://ru.stackoverflow.com/questions/903756/docker-с-контейнером-nginx-не-открывается-сайт>
144. <https://www.linux.org.ru/forum/general/16149430>
145. <https://dockerhosting.ru/blog/nginx-v-docker/>

146. <https://firstvds.ru/technology/ustanovka-i-nastroyka-nginx>
147. <https://django.fun/qa/328152/>
148. <https://ru.stackoverflow.com/questions/1025729/Не-подключается-к-локальному-хосту-через-docker>
149. <https://qna.habr.com/q/951181>
150. https://www.reddit.com/r/node/comments/111xvhu/docker_nginx_localhost_ssl_for_nodejs_reverse/?tl=ru
151. https://www.reddit.com/r/node/comments/111xvhu/docker_nginx_localhost_ssl_for_nodejs_reverse/
152. <https://maxkuznetsov.ru/all/docker-localhost-and-https/>
153. <https://habr.com/ru/companies/cloud4y/articles/547164/>
154. <https://qna.habr.com/q/12232>
155. <https://ru.stackoverflow.com/questions/993177/Не-запускается-html-страница-в-docker>
156. <https://serveradmin.ru/ustanovka-i-nastrojka-nginx/>
157. <https://qa.yodo.im/t/docker-nginx-php-fpm-502-plohoj-shlyuz/1933>
158. https://www.reddit.com/r/docker/comments/1f7h4kh/total_newbie_struggling_with_docker_and_nginx/
159. <https://purpleschool.ru/knowledge-base/article/setup-nginx>
160. https://www.reddit.com/r/docker/comments/kxgoft/issues_dockerizing_nginx_with_node_site_111/
161. <https://labex.io/ru/tutorials/docker-how-to-debug-docker-run-error-when-starting-nginx-container-416181>
162. <https://drupal.ru/node/141974?aura>
163. <https://purpleschool.ru/knowledge-base/article/compose-network>
164. <https://hmarketing.ru/blog/docker/network/>
165. <https://labex.io/ru/tutorials/docker-how-to-create-a-custom-docker-bridge-network-411523>
166. <https://www.tune-it.ru/web/adpashnin/blog/-/blogs/docker-network>
167. <https://habr.com/ru/companies/otus/articles/730798/>
168. <https://labex.io/ru/tutorials/docker-how-to-use-docker-network-create-command-to-manage-container-networks-555174>
169. <https://learn.microsoft.com/ru-ru/virtualization/windowscontainers/container-networking/network-drivers-topologies>
170. <https://docs.docker.com/reference/cli/docker/network/create/>
171. <https://docs.docker.com/engine/network/>
172. <http://onreader.mdI.ru/LearningDockerNetworking/content/Ch02.html>
173. <https://purpleschool.ru/knowledge-base/article/ping>
174. <https://labex.io/ru/tutorials/docker-how-to-test-connectivity-between-docker-containers-411613>
175. <https://hmarketing.ru/blog/docker/network/>
176. <https://purpleschool.ru/knowledge-base/article/get-container-ip>
177. <https://habr.com/ru/articles/790212/>
178. <https://labex.io/ru/tutorials/docker-how-to-diagnose-docker-container-networks-418916>
179. https://www.reddit.com/r/selfhosted/comments/1k3tnls/how_to_monitor_a_docker_containers_network_traffic/
180. <https://habr.com/ru/companies/otus/articles/730798/>

181. <https://itsecforu.ru/2021/03/30/как-проверить-установленные-сетев/>
182. <https://ru.stackoverflow.com/questions/519269/Связь-между-контейнерами>
183. <https://purpleschool.ru/knowledge-base/article/run-docker>
184. <https://arenda-server.cloud/blog/kak-obmenivatsja-dannymi-mezhdu-kontejnerami-docker/>
185. https://www.reddit.com/r/docker/comments/r9k4y0/how_do_i_run_a_container_with_an_ip_on_my_host/
186. <https://journal.sweb.ru/article/docker-na-praktike-polnyy-gayd-po-komandam>
187. <https://ru.stackoverflow.com/questions/1608225/Как-запустить-контейнеры-в-одной-сети>
188. <https://slurm.io/blog/kak-zapustit-kontejner-docker>
189. <https://habr.com/ru/companies/netologyru/articles/967546/>
190. <https://labex.io/ru/tutorials/docker-how-to-configure-a-docker-container-to-use-the-host-network-415202>
191. https://1cloud.ru/help/docker/docker_container
192. https://ru.hexlet.io/courses/docker-basics/lessons/run/theory_unit
193. <https://purpleschool.ru/knowledge-base/article/alpine>
194. https://ru.wikipedia.org/wiki/Alpine_Linux
195. <https://axiomjdk.ru/announcements/2022/08/04/alpine-linux-optimalnyj-distributiv-dlja-docker-obrazov/>
196. <https://core247.kz/blog/container-filesystem>
197. <https://labex.io/ru/tutorials/docker-how-to-run-a-container-based-on-alpine-linux-417725>
198. <https://yandex.cloud/ru/blog/docker-guide>
199. <https://habr.com/ru/companies/netologyru/articles/967546/>
200. <https://ealebed.github.io/posts/2017/3-причины-использовать-alpine-в-качестве-базовых-docker-образов/>
201. <https://practicum.yandex.ru/blog/chto-takoe-docker/>
202. https://www.reddit.com/r/docker/comments/mwkq4g/newbie_question_why_does_linux_docker_need_a/
203. <https://labex.io/ru/tutorials/docker-how-to-handle-container-name-already-in-use-error-in-docker-415733>
204. <https://ru.stackoverflow.com/questions/1076143/При-попытке-старта-селеноид-контейнера-с-новыми-параметрами-получаю-ошибку-что>
205. https://www.reddit.com/r/docker/comments/qrkt3q/the_container_name_is_already_in_use_by_container/
206. <https://purpleschool.ru/knowledge-base/article/container-name>
207. <https://www.linux.org.ru/forum/general/12979529>
208. <https://labex.io/ru/tutorials/docker-how-to-troubleshoot-address-already-in-use-error-in-docker-415734>
209. <https://learn.microsoft.com/ru-ru/sql/linux/sql-server-linux-docker-container-troubleshooting?view=sql-server-ver17>
210. https://www.reddit.com/r/docker/comments/11zgwlu/docker_windows_container_name_already_in_use_by/

211. https://kb.synology.com/ru-ru/DSM/tutorial/How_can_I_debug_and_fix_common_Docker_container_issues

212. <https://habr.com/ru/articles/490882/>

213. <https://purpleschool.ru/knowledge-base/article/volume>

214. <https://timeweb.cloud/tutorials/docker/komandy-docker-spisok>

215. <https://slurm.io/blog/tpost/i5ikrm9fj1-hranenie-dannih-v-docker>

216. <https://journal.sweb.ru/article/docker-na-praktike-polnyy-gayd-po-komandam>

217. https://www.reddit.com/r/docker/comments/1gdvcki/how_to_create_docker_volume/

218. <https://purpleschool.ru/knowledge-base/article/mount>

219. <https://dockerhosting.ru/blog/komandy-docker-shpargalka/>

220. <https://labex.io/ru/tutorials/docker-how-to-create-a-named-docker-volume-414874>

221. <https://habr.com/ru/companies/flant/articles/336654/>

222. <https://cloud.ru/blog/chto-takoye-dockerfile>

223. <https://ru.hexlet.io/qna/sql/questions/postgresql-kak-sozdat-tablitsu>

224. <https://www.8host.com/blog/zaprosy-v-postgresql/>

225. https://sql-ex.ru/blogs/?%2FSozdanie_bazy_dannyh_i_tablic_v_PostgreSQL_Izuchenie_PostgreSQL_s_Grant_Fritchey.html

226. <https://oracleplsql.ru/create-table-as-postgresql.html>

227. <https://docs.arenadata.io/ru/ADPG/current/how-to/queries/psql.html>

228. <https://datafinder.ru/products/postgresql-sozdanie-tablicy>

229. https://ydb.tech/docs/ru/postgresql/statements/create_table

230. <https://postgrespro.ru/docs/postgrespro/10/sql-createtableas>

231. <https://postgrespro.ru/docs/postgrespro/current/tutorial-table>

232. <https://sky.pro/wiki/sql/sozdanie-tablitsy-v-postgresql-esli-ona-esche-ne-suschestvuet/>

233. <https://stackoverflow.com/questions/29599632/container-is-not-running>

234. <https://www.docker.com/blog/how-to-fix-and-debug-docker-containers-like-a-superhero/>

235. <https://community.toradex.com/t/docker-container-registry-not-running/27725>

236. https://www.youtube.com/watch?v=h_sEt1_jRBU

237. https://www.reddit.com/r/docker/comments/ygvjkd/docker_containers_not_starting/

238. <https://forums.docker.com/t/docker-containers-not-running-how-to-fix-it/136752>

239. <https://github.com/docker/for-win/issues/13232>

240. <https://www.youtube.com/watch?v=5EFGHAcXh3c>

241. <https://kodekloud.com/community/t/error-response-from-daemon-container-id-is-not-running/23466>

242. <https://elixirforum.com/t/container-not-running-error-for-docker-exec-command/37500>

243. <https://liupeirong.github.io/postgresSetup/>

244. <https://www.warp.dev/terminus/docker-postgres-container>

245. <https://tsmx.net/postgresql-docker-local-persistent-storage/>

246. <https://github.com/docker-library/postgres/issues/935>

247. <https://github.com/docker-library/postgres/issues/1235>

248. <https://www.docker.com/blog/how-to-use-the-postgres-docker-official-image/>

249. <https://forums.docker.com/t/where-is-my-postgresql-database-path/146043>

250. <https://habr.com/ru/articles/578744/>

251. <https://stackoverflow.com/questions/41637505/how-to-persist-data-in-a-dockerized-postgres-database-using-volumes>

252. https://www.reddit.com/r/unRAID/comments/15ybgg8/where_is_postgres_database_located_when_using/

253. <https://timeweb.cloud/tutorials/postgresql/postgresql-docker-setup>

254. <https://www.warp.dev/terminus/docker-postgres-container>

255. <https://tsmx.net/postgresql-docker-local-persistent-storage/>

256. <https://labex.io/ru/tutorials/docker-how-to-deploy-and-configure-postgres-docker-containers-393104>

257. <https://prohomelab.com/posts/postgres-p-1/>

258. https://help.sweb.ru/ustanovka-i-nastrojka-postgresql-v-docker_1387.html

259. <https://selectel.ru/blog/postgresql-docker-setup/>

260. <https://habr.com/ru/articles/578744/>

261. <https://foxminded.ua/ru/docker-postgres/>

262. https://www.nic.ru/help/polnoe-rukovodstvo-po-ustanovke-i-nastrojke-postgresql-v-docker_11679.html

263. https://www.reddit.com/r/unRAID/comments/pf71dw/setting_up_postgresql_docker_with_multiple/

264. https://alley-science.ru/domains_data/files/EJanuary/METODIKA SOZDANIYa DOCKER-OBRAZA POSTGRESQLV SREDE UBUNTULINUX.pdf

265. <https://foxminded.ua/ru/docker-postgres/>

266. <http://onreader.mdl.ru/DockerRailsDevelopersApplicationsEverywhere/content/Ch06.html>

267. <https://habr.com/ru/articles/823816/>

268. <https://www.cloud4y.ru/blog/installing-and-configuring-postgre-sql/>

269. <https://dockerhosting.ru/blog/postgresql-v-docker/>

270. <https://www.youtube.com/watch?v=BIlWtjJU1Ws>

271. <https://timeweb.cloud/tutorials/postgresql/postgresql-docker-setup>

272. https://help.sweb.ru/ustanovka-i-nastrojka-postgresql-v-docker_1387.html

273. <https://habr.com/ru/articles/578744/>

274. <https://selectel.ru/blog/postgresql-docker-setup/>

275. <https://ru.hexlet.io/qna/sql/questions/postgresql-kak-posmotret-dannye-v-tablitse>

276. <https://ydb.tech/docs/ru/postgresql/statements/select>

277. <https://www.8host.com/blog/zapros-dannyx-postgresql-pri-pomoshhi-komandy-select/>

278. <https://postgrespro.ru/docs/postgresql/12/sql-select>

279. <https://ru.hexlet.io/qna/sql/questions/postgresql-kak-posmotret-tablitsy>

280. <https://postgrespro.ru/docs/postgresql/current/sql-select>

281. <https://docs.arenadata.io/ru/ADPG/current/how-to/queries/psql.html>

282. <https://tproger.ru/translations/useful-postgresql-commands>

283. https://www.reddit.com/r/PostgreSQL/comments/1dgck3d/show_tables_in_current_database_using_psql/

284. <https://habr.com/ru/companies/tensor/articles/780276/>